📊 Scaling Analysis - loan-core Service (Production)

File Analyzed: `values.prd.yaml` (loan-core Helm configuration)
Date: February 5, 2026
Analyst: Senior Cloud Architect / AWS Solutions Expert

🎯 Executive Summary

Component	Horizontal Scaling	Vertical Scaling	Strategy
loan-core Application	✅ **YES** (4-10 pods with HPA)	✅ **YES** (500m CPU, 1GB RAM per pod)	**Hybrid Approach**
RDS MySQL Database	❌ **NO** (No read replicas configured)	✅ **YES** (Instance type upgrades)	**Vertical Only**

Quick Answer (Hinglish)

- **Application:** Dono scaling use ho rahi hai - Horizontal + Vertical ✅
- **Database:** Sirf Vertical Scaling use ho rahi hai ✅

🔷 Part 1: APPLICATION SCALING (loan-core Service)

✅ 1.1 HORIZONTAL SCALING - Fully Implemented

Configuration Evidence

```
autoscaling:
  enabled: true
  minReplicas: 4          # Minimum 4 pods always running
  maxReplicas: 10         # Maximum 10 pods possible
  targetCPUUtilizationPercentage: 60
  targetMemoryUtilizationPercentage: 60
  containerBased: true
```

Kya Ho Raha Hai? (Hinglish Explanation)

Horizontal Scaling ka matlab hai **zyada machines/pods add karna** jab load badhta hai.

- **Normal Time:** 4 pods chalte hain (minReplicas: 4)
- **High Load:** Agar CPU ya Memory 60% cross kar jaye, to automatically **new pods create** ho jate hain
- **Maximum Limit:** 10 pods se zyada nahi ja sakte (maxReplicas: 10)
- **Load Kam Hone Par:** Wapas 4 pods pe aa jate hain

🌟 Real-World Example

McDonald's Counter Analogy:

```
Normal Time (Low Traffic):
👨 👨 👨 👨   →  4 Cashiers (4 pods)

Weekend Rush (High Traffic):
👨 👨 👨 👨 👨 👨 👨 👨 👨 👨   →  10 Cashiers (10 pods)
```

Jab customers ki line lambi hoti hai, manager **extra cashiers** bula leta hai.
Rush khatam hone par wapas **normal staff** pe aa jate hain.

Technical Benefits

- **Auto-scaling:** Manual intervention nahi chahiye
- **High Availability:** Agar 1 pod crash ho, 3+ aur available hain
- **Cost Optimization:** Low traffic pe sirf 4 pods, high traffic pe 10
- **Load Distribution:** Traffic sabhi pods me equally distribute hota hai

---

✅ 1.2 VERTICAL SCALING - Also Configured

Configuration Evidence

```
resources:
  limits:
    cpu: 500m              # 0.5 CPU cores per pod
    memory: 1024Mi         # 1 GB RAM per pod
  requests:
    cpu: 500m
    memory: 1024Mi
```

Kya Ho Raha Hai? (Hinglish Explanation)

Vertical Scaling ka matlab hai **har machine ko zyada powerful banana**.

- **Current Setup:** Har pod ko 500m CPU aur 1GB RAM mila hai
- **Scaling Up:** Agar zarurat ho to increase kar sakte hain:
  - `cpu: 500m` → `cpu: 1000m` (2x power! 💪)
  - `memory: 1024Mi` → `memory: 2048Mi` (2GB RAM! 🚀)

🌟 Real-World Example

Computer Upgrade Analogy:

```
Before (Current):
💻 Basic Laptop (500m CPU, 1GB RAM)  →  Normal speed
```

```
After Vertical Scaling:
🖥️  Gaming PC (2000m CPU, 4GB RAM)  →  Super fast!
```

Yeh cashier ko **basic calculator** se **advanced laptop** dene jaise hai.
Har cashier **zyada fast** kaam kar sakta hai.

Combined Effect (Horizontal + Vertical)

```
Horizontal Scaling:  4-10 pods (more workers)
       +
Vertical Scaling:   500m CPU, 1GB RAM per pod (powerful workers)
       =
💪  BEST OF BOTH WORLDS!
```

Real Scenario:

- **Morning (Low Load):** 4 powerful pods running → Handle normal traffic
- **Afternoon (Peak Load):** 10 powerful pods running → Handle massive traffic
- **Each pod has enough power** to process requests quickly

🔷 Part 2: DATABASE SCALING (RDS MySQL)

❌ 2.1 HORIZONTAL SCALING - NOT Configured

Configuration Evidence

```
envVars:
  MYSQL_HOST: loan-core-mysql-master.backend.prd.idbank-int.net
  MYSQL_HOST_REPLICA: loan-core-mysql-master.backend.prd.idbank-int.net
  DB_NAME: loan_core
```

Problem Kya Hai? (Hinglish Explanation)

Dekho kya gadbad hai:

```
MYSQL_HOST:         loan-core-mysql-master...
MYSQL_HOST_REPLICA: loan-core-mysql-master...  ← Same host! 😱
```

Dono same hain! Matlab:

- ❌ Koi alag **read replica** nahi hai
- ❌ Saare reads aur writes **ek hi database** pe ja rahe hain

- ❌ Horizontal scaling **implement nahi** hui hai

Kya Hona Chahiye? (What Should Be Done)

Proper Horizontal Scaling:

```
envVars:
  MYSQL_HOST: loan-core-mysql-master.backend.prd.idbank-int.net          #
✍ Writes here
  MYSQL_HOST_REPLICA: loan-core-mysql-replica.backend.prd.idbank-int.net #
📖 Reads here
```

🌟 **Real-World Example**

Library Analogy:

Current Setup (Vertical Only):

```
🏛 One Big Library
   ├── People borrowing books (writes)
   └── People reading books (reads)

   Problem: Everyone waiting in same line! 😫
```

Horizontal Scaling (Recommended):

```
🏛 Main Library (Master)
   └── Issue/Return books (writes only)

📚 Branch Library 1 (Read Replica)
   └── Reading only

📚 Branch Library 2 (Read Replica)
   └── Reading only

   Benefit: Multiple locations, faster service! 😊
```

✅ 2.2 VERTICAL SCALING - Available (Implied)

Kaise Hota Hai? (How It Works)

RDS me **instance type change** karke vertical scaling hoti hai.

Example:

```
Current:  db.t3.medium  (2 vCPU, 4GB RAM)    💻
          ↓ Upgrade
Future:   db.r5.xlarge  (4 vCPU, 32GB RAM)   🖥️
          ↓ More Upgrade
Future:   db.r5.4xlarge (16 vCPU, 128GB RAM) 🚀
```

Note: Yeh configuration **AWS RDS console** ya **Terraform** me hoti hai, is Kubernetes file me visible nahi hai.

🌟 Real-World Example

Library Building Upgrade:

```
Small Library:
📖 5 shelves, 2 staff, 10 reading tables

Vertical Scaling (Bigger Library):
📚 50 shelves, 10 staff, 100 reading tables

More Vertical Scaling (Huge Library):
🏛️ 500 shelves, 50 staff, 500 reading tables
```

Same building, but **bigger and more powerful**!

Limitation

Problem: Ek limit ke baad aur bada nahi kar sakte

```
db.r5.24xlarge (96 vCPU, 768GB RAM)  ← Maximum AWS limit

Is se zyada nahi ja sakte! 🛑
Tab horizontal scaling (read replicas) zaruri ho jata hai.
```

📊 Detailed Comparison Table

Application (loan-core) vs Database (RDS)

Aspect	loan-core Application	RDS MySQL Database
Horizontal Scaling	✅ 4-10 pods with HPA	❌ No read replicas
Vertical Scaling	✅ 500m CPU, 1GB RAM per pod	✅ RDS instance upgrades
Auto-scaling	✅ Automatic based on CPU/Memory	❌ Manual intervention needed
Current Approach	🏆 Hybrid (Best practice)	⚠ Conservative (Single instance)

Aspect	loan-core Application	RDS MySQL Database
Scalability Limit	High (10x with HPA, more with resources)	Medium (RDS instance size limit)
Cost Efficiency	✅ Pay only for active pods	⚠ Always pay for full instance
High Availability	✅ Multi-pod redundancy	⚠ Single point of failure risk

---

🔍 Additional Findings

Node Placement Strategy

```
nodeSelector:
  dedicated-graviton: "true"
tolerations:
  - effect: NoSchedule
    key: dedicated
    operator: Equal
    value: defaultGravitonServerGroup
```

Matlab:

- Pods **AWS Graviton processors** pe chalte hain
- Graviton = ARM-based chips (cost-efficient + powerful)
- **Benefit:** 20-40% cost savings compared to x86 instances

Canary Deployment

```
canary:
  enabled: true
  serviceSplit: true
  gatewaySplit: true
  steps:
    - setWeight: 20
    - pause: { }
```

Matlab:

- New version ko **gradually** deploy karte hain
- Pehle 20% traffic pe test karo
- Sab theek hai to 100% pe le jao
- **Safety:** Agar problem ho to quickly rollback

💡 Recommendations

🔴 Critical: Database Horizontal Scaling

Current Problem:

```
MYSQL_HOST_REPLICA: loan-core-mysql-master.backend.prd.idbank-int.net  # Same as master!
```

Recommended Fix:

```
MYSQL_HOST_REPLICA: loan-core-mysql-replica.backend.prd.idbank-int.net  # Separate replica
```

Steps to Implement:

1. **AWS RDS Console:** Create read replica
2. **Update DNS/Endpoint:** Point to new replica
3. **Update Helm values:** Change `MYSQL_HOST_REPLICA` value
4. **Application Code:** Ensure reads go to replica

Benefits:

- ✅ Read load distribute hoga
- ✅ Master pe sirf writes, faster performance
- ✅ High availability (replica failover possible)

🟡 Optional: Increase Pod Resources (If Needed)

Current:

```
resources:
  limits:
    cpu: 500m
    memory: 1024Mi
```

If Performance Issues Exist:

```
resources:
  limits:
    cpu: 1000m      # 2x CPU power
    memory: 2048Mi  # 2x memory
```

When to Do This:

- Agar individual pod **slow** ho
- CPU/Memory limits frequently hit ho rahe ho

- Database queries heavy computation kar rahe ho

Note: Pehle horizontal scaling try karo (cheaper!), phir vertical.

🟢 Nice to Have: Connection Pooling

Add to config:

```
envVars:
  MYSQL_MAX_CONNECTIONS: "100"
  MYSQL_CONNECTION_POOL_SIZE: "20"
```

Benefits:

- Database connections efficiently reuse honge
- Better performance under high load
- Prevents connection exhaustion

📈 Scaling Scenarios

Scenario 1: Normal Business Day

```
Traffic: Low to Medium
├── Pods Running: 4 (minReplicas)
├── CPU Usage: 30-40%
├── Database: Single master handling all traffic
└── Status: ✅ Smooth operation
```

Scenario 2: Peak Hours (Salary Day/Weekend)

```
Traffic: High
├── Pods Running: 7-10 (auto-scaled)
├── CPU Usage: 55-65%
├── Database: Master getting heavy load ⚠
└── Status: ⚠ Database might become bottleneck
```

Recommendation: Add read replicas to prevent database bottleneck!

Scenario 3: Marketing Campaign / Flash Sale

```
Traffic: Very High
├── Pods Running: 10 (maxReplicas - at limit!)
├── CPU Usage: 70-80%
```

```
├── Database: Overloaded! 🔴
└── Status: 🔴 Need to increase maxReplicas + add replicas
```

Action Required:

1. Increase `maxReplicas: 15` (short-term)
2. Add database read replicas (medium-term)
3. Consider caching layer (Redis) (long-term)

🎓 Key Learnings

1. Application Scaling Strategy: Smart! 👍

Why Hybrid Approach Works:

```
Horizontal (4–10 pods) → Handle variable traffic
      +
Vertical (500m/1GB) → Each pod has sufficient power
      =
Cost-effective + High availability + Auto-scaling
```

2. Database Strategy: Conservative ⚠

Current:

- Single master instance (no horizontal scaling)
- Relies on vertical scaling only
- Risk: Single point of failure

Why This Might Be OK (For Now):

- Traffic volume manageable by single instance
- Simpler architecture (no replication lag issues)
- Data consistency easier to maintain

When to Change:

- Read-heavy workload increases
- Master CPU consistently > 70%
- Query response times degrade

🔧 Implementation Checklist

For Database Horizontal Scaling

- ☐ **Step 1:** Monitor current database metrics

    - [ ] CPU utilization
    - [ ] Connection count
    - [ ] Read vs Write ratio

- [ ] **Step 2:** Create RDS read replica
    - [ ] Enable automated backups
    - [ ] Select appropriate instance size
    - [ ] Configure in same VPC

- [ ] **Step 3:** Update application configuration
    - [ ] Change `MYSQL_HOST_REPLICA` endpoint
    - [ ] Test read queries routing
    - [ ] Monitor replication lag

- [ ] **Step 4:** Application code changes (if needed)
    - [ ] Separate read/write connections
    - [ ] Handle replica lag gracefully
    - [ ] Implement retry logic

For Application Vertical Scaling

- [ ] **Step 1:** Analyze current resource usage
    - [ ] Check pod CPU metrics
    - [ ] Check pod memory metrics
    - [ ] Identify bottlenecks

- [ ] **Step 2:** Update Helm values

```
resources:
  limits:
    cpu: 1000m  # Update as needed
    memory: 2048Mi
```

- [ ] **Step 3:** Deploy and monitor
    - [ ] Deploy to staging first
    - [ ] Monitor performance improvements
    - [ ] Roll out to production

📊 Monitoring Metrics

Application Metrics to Watch

```
✅ Pod Count (should auto-scale between 4-10)
✅ CPU Usage per Pod (should stay < 70%)
✅ Memory Usage per Pod (should stay < 80%)
✅ Request Rate (requests/second)
✅ Response Time (p95, p99 latency)
✅ Error Rate (should stay < 5%)
```

Database Metrics to Watch

```
⚠ CPU Utilization (if > 70%, consider read replica)
⚠ Connection Count (if near max, scale up)
⚠ Read Latency (if high, add read replicas)
⚠ Write Latency (if high, upgrade instance)
⚠ Replication Lag (if replicas added)
```

🎯 Final Verdict

What's Working Well ✅

1. **HPA Configuration:** Automatic scaling 4-10 pods is excellent
2. **Resource Allocation:** 500m CPU, 1GB RAM is reasonable per pod
3. **Canary Deployment:** Safe rollout strategy
4. **Graviton Nodes:** Cost-optimized infrastructure
5. **Monitoring:** Prometheus + service monitor enabled

What Needs Improvement ⚠

1. **Database Horizontal Scaling:** No read replicas configured
2. **Replica Host:** `MYSQL_HOST_REPLICA` points to master (incorrect)
3. **Scaling Limits:** Max 10 pods might not be enough for flash sales
4. **Database HA:** Single master is single point of failure

Overall Rating: 7/10 🌟

Strong application scaling, but database needs horizontal scaling for production-grade resilience.

📝 Glossary (Hinglish)

Term	Hinglish Meaning	Example
Horizontal Scaling	Zyada machines add karna	1 server → 10 servers
Vertical Scaling	Ek machine ko powerful banana	2GB RAM → 16GB RAM

Term	Hinglish Meaning	Example
HPA	Horizontal Pod Autoscaler - Automatic pods badhata hai	4 pods → 8 pods (auto)
Replica	Database ki copy (sirf reading ke liye)	Master (write) + Replica (read)
Pod	Kubernetes me chota container (application ka instance)	1 pod = 1 loan-core instance
Canary	Slowly new version deploy karna	Pehle 20%, phir 100%

📚 References

- Kubernetes HPA Documentation: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
- AWS RDS Read Replicas: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html
- Vertical vs Horizontal Scaling: https://www.section.io/blog/scaling-horizontally-vs-vertically/

Document Generated: February 5, 2026
Version: 1.0
Author: Senior Cloud Architect Analysis

🔗 Quick Links

- Application Scaling
- Database Scaling
- Recommendations
- Implementation Checklist

🚦 RATE LIMITER ANALYSIS

❌ Rate Limiter: NOT Configured

Answer: NAHI - Is system me koi **explicit Rate Limiter** configure nahi hai! 🚫

🔍 What I Checked (Investigation Details)

1. ✅ Helm Values File Check

```
# values.prd.yaml me check kiya
- No rateLimit configuration
- No quota settings
- No throttling parameters
```

2. ✅ Istio Configuration Check

```
# VirtualService me check kiya
- Basic routing rules only
- No rate limiting policies
- No connection limits
```

3. ✅ DestinationRule Check

```
# DestinationRule me check kiya
- Only canary/stable subsets defined
- No connection pool limits
- No circuit breaker rate limits
```

4. ✅ Application Config Check

```
# config.json me check kiya
- MySQL circuit breaker hai (but that's different)
- No API rate limiting configuration
- No request throttling
```

🎯 5 Types of Rate Limiters (Brief Overview)

Type	Kaise Kaam Karta Hai	Example
1. Token Bucket	Bucket me tokens hote hain, har request 1 token consume karti hai	AWS API Gateway default
2. Leaky Bucket	Fixed rate se requests process hoti hain, overflow discard	Network traffic shaping
3. Fixed Window Counter	Fixed time window me X requests allowed	100 requests per minute
4. Sliding Window Log	Har request ka timestamp track karta hai	Precise rate limiting
5. Sliding Window Counter	Fixed + Sliding ka hybrid approach	Memory efficient + accurate

---

⚠ Current Situation (Kya Ho Raha Hai)

Without Rate Limiter:

```
Client → API Gateway → Istio → loan-core pods (4-10)
         ❌ No rate limit    ❌ No limit    ✅ HPA handles load
```

Risk:

- ❌ Single client **unlimited requests** bhej sakta hai
- ❌ **DDoS attack** ka risk
- ❌ **Abuse** ho sakta hai (koi bhi 1000s requests bhej sakta hai)
- ⚠ Sirf HPA pe dependent (but HPA slow react karta hai)

🌟 Real-World Analogy (Hinglish)

Current Setup (No Rate Limiter):

```
🏪 McDonald's Counter
└── Koi bhi customer kitne bhi orders de sakta hai
    └── 1 customer = 100 burgers order kiya
    └── Other customers = Wait kar rahe hain 😫
    └── Manager = More staff bulata hai (HPA) - but late!
```

With Rate Limiter (Recommended):

```
🏪 McDonald's Counter with Rules
├── Rule: Max 5 orders per customer per minute (Token Bucket)
├── Fair access for all customers ✅
└── No single customer can overwhelm the counter ✅
```

💡 Recommendations: Rate Limiter Add Karo!

🔴 Option 1: Istio EnvoyFilter (Recommended for Kubernetes)

Configuration Example:

```yaml
apiVersion: networking.istio.io/v1beta1
kind: EnvoyFilter
metadata:
  name: loan-core-rate-limit
  namespace: backend-idbank-app-01
spec:
  workloadSelector:
    labels:
```

```
      app.kubernetes.io/name: loan-core
  configPatches:
  - applyTo: HTTP_FILTER
    match:
      context: SIDECAR_INBOUND
    patch:
      operation: INSERT_BEFORE
      value:
        name: envoy.filters.http.local_ratelimit
        typed_config:
          "@type": type.googleapis.com/udpa.type.v1.TypedStruct
          type_url:
type.googleapis.com/envoy.extensions.filters.http.local_ratelimit.v3.Local
RateLimit
          value:
            stat_prefix: http_local_rate_limiter
            token_bucket:
              max_tokens: 100          # 100 requests
              tokens_per_fill: 100
              fill_interval: 60s       # Per minute
            filter_enabled:
              runtime_key: local_rate_limit_enabled
              default_value:
                numerator: 100
                denominator: HUNDRED
```

Type: Token Bucket 🪣 Benefits:

- ✅ Per-pod rate limiting
- ✅ No external dependencies
- ✅ Fast (runs in Envoy sidecar)
- ✅ Simple to configure

🟡 Option 2: Application-Level Rate Limiter

Go Code Example (Using Redis):

```go
// Using Token Bucket with Redis
import "github.com/go-redis/redis_rate/v10"

limiter := redis_rate.NewLimiter(redisClient)
res, err := limiter.Allow(ctx, "loan-core:api", redis_rate.PerMinute(100))

if err != nil {
    return err
}

if res.Allowed == 0 {
    return errors.New("Rate limit exceeded")
}
```

Type: Token Bucket or Sliding Window Counter (depends on library) **Benefits:**

- ✅ Fine-grained control per endpoint
- ✅ Per-user rate limiting possible
- ✅ Flexible business logic
- ⚠ Requires Redis (you already have it!)

🟢 Option 3: API Gateway Rate Limiting

If using AWS API Gateway or Kong:

```
# API Gateway throttling
- Burst Limit: 500 requests
- Rate Limit: 1000 requests per second
- Per API Key limits
```

Type: Token Bucket (AWS default) **Benefits:**

- ✅ Centralized control
- ✅ Protects entire backend
- ✅ Easy to configure
- ⚠ Need API Gateway in place

📊 Recommended Rate Limits (Production Best Practices)

For loan-core Service:

Endpoint Type	Suggested Limit	Type	Reason
Public APIs	100 req/min per IP	Token Bucket	Prevent abuse
Internal APIs	500 req/min per service	Sliding Window Counter	Allow burst traffic
Critical APIs (loan creation)	10 req/min per user	Leaky Bucket	Prevent fraud
Health Check	Unlimited	-	No limit needed
Metrics	Unlimited	-	Internal monitoring

🔧 Implementation Priority

Phase 1: Quick Win (1-2 days)

```
✅ Add Istio EnvoyFilter with Token Bucket
   - Global rate limit: 1000 requests/min per pod
```

```
   - Quick deployment
   - Immediate DDoS protection
```

Phase 2: Per-Endpoint Control (1 week)

```
✅ Application-level rate limiting
   - Use existing Redis
   - Per-endpoint limits
   - Per-user tracking
```

Phase 3: Advanced (2-4 weeks)

```
✅ Custom rate limiting logic
   - Business-specific rules
   - Dynamic limits based on customer tier
   - Monitoring & alerting
```

🎓 Circuit Breaker vs Rate Limiter (Confusion Mat Hona!)

Is config me Circuit Breaker hai, but that's different:

```
"masterCircuitBreaker": {
  "maxConcurrentReq": 5000,      # ← This is NOT a rate limiter!
  "errorPercentThreshold": 50,
  "sleepWindowInMs": 5000
}
```

Difference:

Aspect	Circuit Breaker (Available ✅)	Rate Limiter (Not Available ❌)
Purpose	Protect downstream services	Control incoming traffic
Trigger	Error rate/timeouts	Request count/time
Action	Stop calls if service failing	Reject excess requests
Example	MySQL failing → stop queries	Too many API calls → reject
In Config?	✅ YES (MySQL circuit breaker)	❌ NO

🚨 Why Rate Limiter is Critical for Production

Real Incident Examples:

Scenario 1: Bug in Client Code

```
❌ Without Rate Limiter:
Client has infinite loop → 10,000 requests/sec
→ All 10 pods overwhelmed
→ Service down for everyone! 🔴

✅ With Rate Limiter:
Client hits limit → 100 requests blocked
→ Other clients unaffected
→ Service stable! ✅
```

Scenario 2: DDoS Attack

```
❌ Without Rate Limiter:
Attacker floods API → Pods scale to 10
→ Database overwhelmed
→ Cascading failure! 🔴

✅ With Rate Limiter:
Attack traffic rejected at edge
→ Backend protected
→ Legitimate users served! ✅
```

📈 Monitoring Metrics (After Adding Rate Limiter)

Add these metrics:

```
✅ rate_limit_requests_total (counter)
✅ rate_limit_requests_rejected (counter)
✅ rate_limit_tokens_available (gauge)
✅ rate_limit_clients_blocked (counter)
```

Alert on:

```
⚠ High rejection rate (> 10%) → Legitimate traffic issue
⚠ Single IP many rejections → Potential attack
⚠ Sudden spike in rate limit hits → Investigate
```

🎯 Final Answer on Rate Limiter

Current Status:

- ❌ **NO Rate Limiter** configured
- ⚠ System relies only on HPA (reactive, not preventive)
- ⚠ Vulnerable to API abuse/DDoS

Recommendation:

- 🔴 **High Priority:** Implement Token Bucket via Istio EnvoyFilter
- 🟡 **Medium Priority:** Add application-level rate limiting with Redis
- 🟢 **Nice to Have:** API Gateway throttling (if applicable)

Type to Implement:

For loan-core, best choice is **Token Bucket** because:

- ✅ Allows burst traffic (good for banking)
- ✅ Simple to configure
- ✅ Industry standard
- ✅ Works with Istio out-of-the-box

📝 Updated Summary Table

Feature	Status	Implementation
Horizontal Scaling	✅ Configured	HPA (4-10 pods)
Vertical Scaling	✅ Configured	500m CPU, 1GB RAM per pod
Rate Limiter	❌ **NOT Configured**	**Need to add!**
Circuit Breaker	✅ Configured	MySQL circuit breaker
Load Balancing	✅ Configured	Istio (automatic)
Auto Scaling	✅ Configured	HPA on CPU/Memory

Document Updated: February 5, 2026
Version: 1.1
Author: Senior Cloud Architect Analysis
New Section Added: Rate Limiter Analysis

End of Document